

Alpha Build Plan

Your people. Your language.

Prepared by: Tony Forbes

Document Version: 0.4 (Draft for Review)

Date: 4 November 2026

Classification: Confidential

Document Control

Version History

Version	Date	Author	Description
0.1	4 November 2026	Tony Forbes	First version: alpha decisions (no end-to-end encryption; chat and translation, groups and tasks, events and the strip, family tree, Find and Play), demo scripts, what is simplified, alpha architecture and data model, security rules, seed data, milestones with acceptance tests, setup checklist and CLAUDE.md.
0.2	4 November 2026	Tony Forbes	Translation proxy and matching service built as the real Cloud Run services; the server never reads message text (phones translate, write translations and create cards); new 4.1 Cloud Run services, 4.2 endpoints and 4.3 triggers; chats and events created only by endpoints; usage and match collections; warm instances for demos; tests and CLAUDE.md updated.
0.3	4 November 2026	Tony Forbes	Private data follows the Data Model's private plane: every task is a message (group chat, 1:1 chat or the user's private My tasks chat), and tasks can now be shared in 1:1 chats; private events, game invites, Find request details and quotes are messages; statuses through packages/crypto. Server tasks collection and private events removed; events.create and events.invite removed; requests hold no details; rules, seed data, tests and CLAUDE.md updated; open question on moving history to a new phone.
0.4	4 November 2026	Tony Forbes	Message yourself added (each account's own chat, pinned at the top of Chats, as in WhatsApp); personal tasks live there, replacing the My tasks chat; task updates there show on the card, not as lines; tests and seed data updated. Aligned with Functional Specification v0.13, Technical Architecture v0.11 and Data Model v0.12.

Related Documents

Document	Description	Status
Yttr Functional Specification	Functional behaviour the alpha follows	Version 0.13 (draft)
Yttr Technical Architecture	Full platform the alpha simplifies	Version 0.11 (draft)
Yttr Data Model	Full data model the alpha uses a subset of	Version 0.12 (draft)
Yttr interactive prototype	Clickable prototype the alpha matches	Version 60

Distribution

This document is confidential. It is shared with the Yttr product and engineering team and with the reviewers named in Section 13.

Table of Contents

1. Purpose and Decisions.....4
2. Demo Scripts.....5
3. What Is Real and What Is Simplified.....6
4. Alpha Architecture.....8
    4.1 Cloud Run Services.....8
    4.2 Endpoints (Callable Functions).....8
    4.3 Triggers and Scheduled Jobs.....9
5. Alpha Data Model.....11
6. Security Rules.....13
7. Seed Data and Demo Accounts.....15
8. Milestones and Acceptance Tests.....16
9. Demo Readiness.....18
10. Tony's Setup Checklist.....19
11. Claude Code Setup.....20
12. Risks and Open Questions.....21
13. Approval and Sign-off.....22

1. Purpose and Decisions

The alpha is a working Yttr on real phones, used by about 20 invited people, for presentations, demos and investor discussions. It follows the Yttr Functional Specification v0.13, Technical Architecture v0.11 and Data Model v0.12, with the simplifications in section 3, so that it can be built quickly with Claude Code. Every simplified part keeps the interface of the full design, so the pilot build replaces parts without rewriting the app.

Decided (4 November 2026)

- No end-to-end encryption in the alpha. Messages are protected by Google's encryption at rest and by strict access rules; the pilot build adds vodozemac end-to-end encryption behind the same crypto interface.
- Included: chat and translation, groups and tasks, events and the Local deals strip, the family tree, Find and Play.
- Not included: Yttr Plus, payments, paid promotion, in-app voice and video calls.
- Translation is unlimited for alpha users.
- Translation and matching run as the real Cloud Run services of the full design: the translation proxy and the matching service (ADR-12). The key directory is not built in the alpha.
- The server never reads message text. Phones translate through the proxy, write the original and its translations together, and create task, quote and invite cards themselves. Adding end-to-end encryption later then changes only packages/crypto, the key directory and the inbox, not the features.
- Private data follows the Data Model's private plane: tasks, private events and game invites, guest lists, Find request details and quotes are messages, never server collections. Statuses are written through packages/crypto.
- Every task is a message: group tasks in the group chat, shared tasks in a 1:1 chat (new: a task can be assigned to the other person in a 1:1 chat), and personal tasks in the user's own Message yourself chat, linked to a related chat when there is one. The phone builds the Tasks tab and group boards from these messages.
- Message yourself (new, as in WhatsApp): every account has its own chat, pinned at the top of Chats as "Name (You)", for notes, links, photos, voice notes and personal tasks. In this chat, task updates are not shown as separate lines: the task card shows the latest status. Nothing in it is translated, because the only reader is the writer.

2. Demo Scripts

Each script must run reliably on two or three phones. They are the top-level acceptance tests for the alpha. Phone A is Tony (English), B is Thandi (isiZulu), C is Pieter (Afrikaans), D is a service provider.

#	Demo	Steps	Must show
1	Talk across languages	B sends a message in isiZulu to A; A replies in English	A reads English with “Translated from isiZulu” within 2 seconds; B reads isiZulu; Show original works on both
2	Family group	In the Family group (A, B, C), C writes in Afrikaans	Each member reads their own language; the sender’s “See what they see” lists every version
3	Group task	A turns a message into a task, assigns C, due Saturday	C sees a translated task card in the chat and on the Tasks tab; marking it done posts to the chat
4	Event for you	A has Stormers as a team	The strip shows “Stormers home match · For you”; tap, then Going; the event appears under Going; switching off Teams removes it from the strip
5	Local deal	The strip rotates to Uncle Tony’s	Tap opens the deal with Directions, Call, Message and Save; Hide this advertiser removes it
6	Family tree	A opens Family, Mom’s side	A adds a contact as a sister; kinship words switch to isiZulu; A messages a relative from their page
7	Find	A requests a pool clean	D receives the request in their language, sends a quote card; A accepts and a task is created
8	Play	A invites C to padel on Saturday at 10:00	C accepts in the chat; the game appears under Going for both
9	Language switch	A changes preferred language to Afrikaans	New messages, the strip and event titles appear in Afrikaans

3. What Is Real and What Is Simplified

Area	Alpha	Full design later
Sign-in	Firebase phone sign-in; a blocking function allows only numbers on the alpha list	Pilot list, waiting list, member suggestions
Messages	Stored in Firestore under each chat, readable only by its members (security rules), encrypted at rest by Google; the phone writes the original and its translations through packages/crypto, whose alpha version passes them through; the server never reads message text	vodozemac Olm and Megolm, key directory, encrypted inbox
Translation	The real translation proxy: the sender's phone calls it once per message with the members' languages; it checks each reader's entitlement (allowance switched off, so unlimited), counts usage and stores no text; the Translate tap calls it from the reader's phone; one consent notice	Allowance and Yttr Plus switched on, Private translation, offline fallback
Voice notes, photos	Cloud Storage, readable by chat members; no transcripts	Encrypted media
Status	Text statuses to the poster's chosen audience, translated by the poster's phone, written through packages/crypto, gone after 24 hours	Encrypted status payloads to each contact's inbox
Message yourself	The user's own chat, pinned at the top of Chats, with every message kind; personal tasks live here	Same, encrypted
Groups and tasks	As spec sections 4 and 6, up to 100 members; every task is a message (group chat, 1:1 chat or Message yourself) and the phone builds the boards	Same, encrypted
Events	Community events loaded by script, with RSVP on the server; private events and game invites are messages in the chat, and the phone builds the guest list	Organiser portal and staff tools
Local deals strip	As spec 13.4.1, from the real matching service (ADR-12): the phone keeps its pseudonymous match profile up to date and pulls its items every 4 hours	Paid ads shared by budget
Family	Own tree; link contacts; kinship words in English, Afrikaans and isiZulu; Family now	Tree matching and merging across families
Find	Seeded providers, contacts' providers first, then nearby; the request details, quote and acceptance are messages in the stranger chat	Trust tiers, Typesense search
Play	Invites to contacts and groups	Nearby players, open games
Search	Firestore queries only	Typesense Cloud
Analytics	Crashlytics only	Firebase Analytics and BigQuery

Area	Alpha	Full design later
Notifications	Push for messages, invites, tasks, quotes	Full rules in spec section 10
Distribution	TestFlight (iPhone) and Google Play internal testing, built with EAS; the prototype link for everyone else	App stores

Left out of the alpha: Yttr Plus, payments, paid promotion, calls, Private translation, the web portal, web RSVP pages, staff tool screens (scripts are used instead), Download my data (done by hand on request).

4. Alpha Architecture

- **Projects:** two Firebase projects, yttr-alpha-dev for building and yttr-alpha for demos. Firestore, Cloud Storage and Cloud Functions run in africa-south1; Cloud Translation is a global Google service.
- **App:** Expo with a development client, React Native and TypeScript, Expo Router for screens, React Native Firebase for sign-in, Firestore, Storage, push and Crashlytics. Firestore's offline cache is used instead of SQLite in the alpha.
- **Shared code:** packages/shared holds Zod schemas and types for every document and message kind; packages/crypto holds the crypto interface with the pass-through alpha implementation; packages/i18n holds interface text in English, Afrikaans, isiZulu, Portuguese and French.
- **Server:** two Cloud Run services in Node.js and TypeScript, built as in Technical Architecture 9.3 and 9.6: the translation proxy and the matching service. Everything else is Cloud Functions (2nd generation), also Node.js and TypeScript, which Firebase deploys onto Cloud Run.
- **No message text on the server:** no function or service reads message text or captions. The translation proxy sees text only while translating and stores none. Push notifications carry no content ("New message from Thandi"); the phone shows the detail.
- **Warm start:** the translation proxy, the matching service and onMessageCreate keep one instance running (minInstances 1), so the first message after a pause is not delayed by a cold start of 1 to 3 seconds.

4.1 Cloud Run Services

Service	Route	Input	Returns
Translation proxy	POST /translate	text, from (optional), to (languages); for messages also chatId and readers (uid for each language); Firebase sign-in token and App Check. Statuses, about lines and event text are translated the same way, into all five languages	translations by language, detected language, languages not translated (none in the alpha)
Matching	PUT /match/{key}/profile	key secret; tags, suburb, visiting place, adult, interestAds, hiddenAdvertisers, tagWeights; App Check, no user identity	ok
Matching	GET /match/{key}	key secret	up to five items (events for you and ads), computedAt
Matching	POST /match/{key}/action	key secret; item, action (shown, tap, save, hide, report)	ok

The matching service recomputes every profile's items every 4 hours (Cloud Scheduler), and at once after a profile change.

4.2 Endpoints (Callable Functions)

Endpoints are used only where the server must act: lookups across users, changes to other people's data, and notifications. Everything else is read and written in Firestore directly under the security rules.

Endpoint	Input	Returns	Why it is on the server
beforeCreate (blocking)	phone number	allow or deny	Checks the alpha list
account.setLanguage	lang	ok	Updates memberLangs in every chat of the user
account.registerDevice	push token, platform	ok	Keeps push tokens out of documents other users can read
account.delete	none	ok	Removes the user's data across collections
contacts.match	hashed phone numbers (up to 500)	those on Yttr, with uid and name	Users cannot read each other's numbers; rate-limited
chats.openDirect	otherUid	chatId	One chat per pair, with a fixed ID, so two phones never create two chats
chats.createGroup	name, memberUids	chatId	Enforces the 100-member cap and fills memberLangs
chats.addMembers, chats.removeMember, chats.leave	chatId, uids	ok	Changes other people's access; records the change on the chat (not as a message)
events.rsvp	community eventId, status (going, maybe, no)	counts	Counts for community events; each user changes only their own status
find.search	skill, suburb	ranked providers	Contacts' providers first, which needs the contact graph
find.request	providerUid, skill	reqId, chatId	Opens the stranger chat with translation opt-in and records the request without its details; the phone sends the details as a message

Tasks, task updates, private event and game invites, RSVPs to them, Find request details, quotes and acceptances are message kinds sent by the phones; no endpoint handles them.

4.3 Triggers and Scheduled Jobs

Function	Trigger	What it does
onUserCreate	New account	Creates the user's Message yourself chat
onMessageCreate	New document in chats/{chatId}/messages	Updates the chat's last activity time and sender (never content) and pushes "New message from …" to the other members
onRsvpWrite	Community event RSVP changed	Updates counts
onRequestWrite	Find request opened or its status changed	Notifies the other person
cleanup	Nightly	Removes expired requests and erases match profiles unused for 60 days

Staff scripts (not callable from the app): addAlphaUser, loadEvents, loadAds, seed, reset.

5. Alpha Data Model

A subset of Data Model v0.12, with plain (not encrypted) messages. Everything on the Data Model's private plane exists only as messages or in the user's own Message yourself chat; no server collection holds tasks, private events, guest lists, request details or quotes.

Collection	Main fields	Who can read
users/{uid}	phone, displayName, lang, settings (autoTr, suggest, privacy, pushOn), createdAt	The user
usage/{uid}/days/{date}	translated characters received and Translate taps, written by the translation proxy	The user
profiles/{uid}	displayName, initials, colour, lang, suburbId, about (orig and translations), interests, skills, teams, food, birthday, contact details	Signed-in alpha users
contacts/{uid}/list/{otherUid}	name, addedAt	The user
chats/{chatId}	type (direct, group, stranger, self), members, memberLangs, name, createdBy, lastAt, lastFrom (no text), membership changes	Members
chats/{chatId}/messages/{msgId}	from, kind (text, voice, image, task, task_update, event_invite, game_invite, rsvp, request, quote, quote_accept, location, contact, status_reply), orig (language), text, t (translations by language, written by the sender's phone), payload, createdAt	Members
statuses/{uid}/items/{id}	text with translations (through packages/crypto), audience (uids), expiresAt	The poster and the audience
events/{eventId}	community events only: title and about (orig and translations), startsAt, place, suburbId, tags, organiser	Signed-in alpha users
events/{eventId}/rsvps/{uid}	status	The user
ads/{adId}	advertiser (name, icon), headline, sub and body (orig and translations), place, km, validUntil, phone, targeting (suburbs, tags), schedule, maxImpressions, active	Signed-in alpha users
family/{ownerUid}/persons/{pid} and family/{ownerUid}/links/{lid}	As Data Model 6.5, one tree per owner	The owner
providers/{uid}	skills, suburbId, about, response time	Signed-in alpha users
requests/{reqId}	from, providerUid, skill, chatId, status (open, quoted, accepted, closed); no details or amounts	Requester and provider
tags/{tagId}	kind, t (names by language)	Signed-in alpha users
matchProfiles/{key} and matchResults/{key}	As Data Model 7.4; no name, phone or uid	Nobody (matching service only)

Collection	Main fields	Who can read
alphaList/{phoneHash}	addedAt, note	Nobody (functions only)

6. Security Rules

These rules are the starting point for firestore.rules. Claude Code writes emulator tests for each rule before building the screen that depends on it.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    function signedIn() { return request.auth != null; }
    function me() { return request.auth.uid; }
    function chat(c) { return get(/databases/$(db)/documents/chats/$(c)).data; }
    function isMember(c) { return signedIn() && me() in chat(c).members; }
    match /users/{uid} {
      allow read, write: if signedIn() && me() == uid;
    }
    match /profiles/{uid} { allow read: if signedIn(); allow write: if signedIn() && me() == uid;
}
    match /contacts/{uid}/list/{other} { allow read, write: if signedIn() && me() == uid; }
    match /chats/{c} {
      allow read: if signedIn() && me() in resource.data.members;
      allow create, delete: if false;
      allow update: if signedIn() && me() in resource.data.members && request.resource.data.diff(r
esource.data).affectedKeys().hasOnly(['name']);
      match /messages/{m} {
        allow read: if isMember(c);
        allow create: if isMember(c) && request.resource.data.from == me();
        allow update, delete: if false;
      }
    }
    match /statuses/{uid}/items/{s} {
      allow read: if signedIn() && (me() == uid || me() in resource.data.audience);
      allow create, update, delete: if signedIn() && me() == uid;
    }
    match /events/{e} {
      allow read: if signedIn();
      allow write: if false;
      match /rsvps/{uid} { allow read: if signedIn() && me() == uid; allow write: if false; }
    }
    match /providers/{uid} { allow read: if signedIn(); allow write: if signedIn() && me() == uid;
}
    match /requests/{r} {
      allow read: if signedIn() && me() in [resource.data.from, resource.data.providerUid];
      allow update: if signedIn() && me() in [resource.data.from, resource.data.providerUid] && re
quest.resource.data.diff(resource.data).affectedKeys().hasOnly(['status']);
      allow create, delete: if false;
    }
    match /family/{owner}/{coll}/{id} { allow read, write: if signedIn() && me() == owner; }
    match /ads/{a} { allow read: if signedIn(); allow write: if false; }
    match /tags/{t} { allow read: if signedIn(); allow write: if false; }
    match /usage/{uid}/days/{d} { allow read: if signedIn() && me() == uid; allow write: if false;
}
    match /matchProfiles/{k} { allow read, write: if false; }
    match /matchResults/{k} { allow read, write: if false; }
    match /alphaList/{h} { allow read, write: if false; }
  }
}
```

Chats and Find requests are created only by endpoints and triggers, community events only by staff scripts, and RSVPs to community events only through events.rsvp. Storage rules follow the same membership test: chat media lives under chats/{chatId}/ and is readable and writable only by members; profile photos under profiles/{uid}/ are readable by signed-in users and writable by the owner.

7. Seed Data and Demo Accounts

- **People** (from the prototype): Tony Forbes (English, Durbanville; team Stormers; interests braai, hiking, live jazz), Thandi Nkosi (isiZulu), Pieter van Wyk (Afrikaans), Lucas Ferreira (Portuguese), Amélie Laurent (French), and Green Pool Care as a provider.
- **Demo accounts** use Firebase test phone numbers with fixed verification codes, so sign-in works on stage without waiting for an SMS. Invited alpha users sign in with their real numbers.
- **Chats:** Tony and Thandi (1:1), Stormers Supporters (Tony, Pieter, Lucas, Amélie), Family (Tony, Thandi, Pieter).
- **Events:** Stormers home match (DHL Stadium), Lion's Head sunrise hike, Jazz evening in Durbanville, and a family birthday dinner sent as an invite message in the Family chat. The seed script sets dates relative to the day it runs, so events are always in the coming week.
- **Tasks:** a group task in Stormers Supporters, a shared task in the Tony and Thandi chat, and two personal tasks and a saved link in Tony's Message yourself chat, all as messages.
- **Ads:** Uncle Tony's Shisa Nyama (R90 buy-1-get-1-free braai), Green Pool Care (first clean R350), and Yttr notices.
- **Family tree:** the Petersen and Adams families from the prototype, with great-grandparents, and Thandi linked as a relative.
- **Tags:** a starting list of about 40 interests, skills, teams and foods in all five languages, to be reviewed by Tony.
- **Reset:** a script restores the seed state before each demo.

8. Milestones and Acceptance Tests

Each milestone ends with a build installed on two phones and the listed tests passing. Tests are written as Given / When / Then and automated where possible (emulator tests for rules and functions, Maestro flows for the app).

M0 · Foundations

1. Given the repository, when CI runs, then lint, type checks and unit tests pass.
2. Given the emulators, when the seed script runs, then all seed documents exist.
3. Given EAS, when a build is made, then it installs on an iPhone (TestFlight) and an Android phone (internal testing).
4. Given a deploy, when the translation proxy and the matching service are called, then both answer from africa-south1 with one warm instance each.

M1 · Sign-in, profile, contacts

1. Given a number on the alpha list, when it signs in, then the account is created and the profile setup opens.
2. Given a number not on the list, when it signs in, then it sees “Yttr is in a private pilot” and no account is created.
3. Given the profile, when the user sets language, suburb, interests and teams, then they are saved and shown on the profile.

M2 · Chat and translation (demo 1)

1. Given A reads English and B reads isiZulu, when B sends “Sawubona! Unjani?”, then A sees the English translation labelled “Translated from isiZulu” within 2 seconds.
2. Given a translated message, when A taps Show original, then the isiZulu text appears.
3. Given “Meet at 10:00 at 12 Main Road, R350”, when it is translated, then 10:00, 12 Main Road and R350 are unchanged.
4. Given A is offline, when A sends a message, then it shows as pending and is translated and delivered when A is back online.
5. Given a message in a group with isiZulu, English and Afrikaans readers, when it is sent, then the sender’s phone calls the translation proxy once with English and Afrikaans, and the stored message holds the original and both translations.
6. Given the server code, when it is reviewed, then no function or service reads message text or captions, and push notifications show only the sender’s name.

M3 · Groups, tasks, voice notes, photos, status (demos 2 and 3)

1. Given a group with English, isiZulu and Afrikaans readers, when one member writes, then each reads their own language and the sender sees all versions in See what they see.
2. Given a message in the group, when A creates a task from it and assigns C, then A’s phone sends a task message, and C sees the card in the chat and the task on the Tasks tab, in Afrikaans.
3. Given the task, when C marks it done, then a task_update message shows the change to all members, and a member who was offline sees it on reconnecting.
4. Given the 1:1 chat with B, when A assigns “pick up the cake” to B, then the task appears in both Tasks tabs.
5. Given a message in the 1:1 chat with B, when A makes it a personal task, then a task card appears only in A’s Message yourself chat, linked to that chat, and B sees nothing.
6. Given that personal task, when A marks it done on the Tasks tab, then its card in Message yourself shows it done and no new line is added.

7. Given the tasks feature in use, when the server data is inspected, then no collection outside chat messages holds a task title.
8. Given a voice note or photo, when it is sent, then members can play or view it; a caption is translated.

M4 · Events and the Local deals strip (demos 4 and 5)

1. Given A has team Stormers, when A's phone pulls GET /match/{key}, then the Stormers home match is the first item, labelled For you.
2. Given A switches off Teams in Suggest events and deals from, when the phone updates its match profile, then the match is no longer in the strip.
3. Given the matchProfiles collection, when it is inspected, then no profile holds a name, phone number or uid.
4. Given the strip, when it rotates, then it shows at most five items, one per advertiser, changing every 6 seconds, with dots and swipe.
5. Given A taps Uncle Tony's, when the deal opens, then Directions, Call, Message, Save, Why am I seeing this?, Hide and Report are shown; Hide removes the advertiser.
6. Given A creates a private event and invites the Family group, when B replies Going, then A sees B under Going.

M5 · Family (demo 6)

1. Given A's tree, when A adds Thandi from contacts as a sister, then she appears on the right side with the right kinship word in A's chosen language.
2. Given the kinship language is isiZulu, when A views Mom's side, then umalume, ubabekazi and the other words appear as in the spec.
3. Given a relative's page, when A taps Message, then the chat opens.

M6 · Find, Play and demo readiness (demos 7, 8 and 9)

1. Given D offers pool cleaning, when A sends a request, then D receives it in D's language and can send a quote card.
2. Given the quote, when A accepts, then a quote_accept message and a shared task appear in the stranger chat for both, and the request status shows accepted.
3. Given A invites C to padel on Saturday at 10:00, when C accepts, then the game appears under Going for both.
4. Given A changes preferred language to Afrikaans, when new messages arrive, then they are shown in Afrikaans.
5. Given the reset script, when it runs, then the demo accounts are back to the seed state in under a minute.

9. Demo Readiness

- Run demos on the yttr-alpha project with the demo accounts, after the reset script.
- Translation needs a connection: carry a mobile hotspot and send one message on each phone before the demo starts.
- Keep the clickable prototype ready for screens the alpha does not have (web portal, Yttr Plus, calls).
- **On security, say:** “The alpha keeps messages in Google’s encrypted storage with strict access rules. The pilot build adds end-to-end encryption, so Yttr itself cannot read messages.”
- Real alpha users must be told the alpha is not end-to-end encrypted, in the consent notice at sign-up.

10. Tony’s Setup Checklist

Item	Why	Needed by
Google Cloud billing account and two Firebase projects (yttr-alpha-dev, yttr-alpha)	Everything runs there	M0
Cloud Translation API enabled, with a budget alert	Translation	M0
Apple Developer Program (company enrolment needs a D-U-N-S number and can take days)	TestFlight builds for iPhone	M0
Google Play Console	Internal testing for Android	M0
Expo account	EAS builds	M0
Private GitHub repository	Code and CI	M0
An iPhone and an Android phone for demos, plus one more phone	Demos with three people	M1
The alpha list of about 20 phone numbers	Sign-in	M1
Short alpha privacy notice and consent text (POPIA), stating that the alpha is not end-to-end encrypted	Sign-up	M1
Launch partners’ consent, logos and offers (Uncle Tony’s and one or two others)	Local deals strip	M4
Review of the starting tag list and of seed names and photos (fictional or with consent)	Seed data	M4

11. Claude Code Setup

The repository root holds this CLAUDE.md. Claude Code works one milestone at a time: it turns each acceptance test into a GitHub issue, writes the test first, builds until it passes, and stops at the end of each milestone for Tony to try the build on a phone.

```
# CLAUDE.md · Yttr alpha
Read docs/alpha-build-plan.md first, then the spec, architecture and data model in docs/.
Build one milestone at a time (M0 to M6). Stop at the end of each milestone for review.
Stack: Expo (dev client), React Native, TypeScript, Expo Router, React Native Firebase; Cloud Run
services translate/ and match/ and Cloud Functions 2nd gen, all Node.js and TypeScript; region afr
ica-south1.
Every Firestore document and message kind has a Zod schema in packages/shared; never write untyped
data.
All message text goes through packages/crypto (pass-through in the alpha); never bypass it.
Write emulator tests for every security rule and function before the screen that uses them.
Interface text lives in packages/i18n in en, af, zu, pt and fr; never hard-code user-facing text.
Translation goes only through the translation proxy (services/translate); the app never calls Goog
le directly and holds no API keys.
The server never reads message text: no function or service may read message text or captions; the
proxy translates and stores nothing.
The matching service never receives a name, phone number or uid.
Private things are messages: tasks, private events, game invites, RSVPs to them, Find request deta
ils and quotes are message kinds; never add a server collection for them.
The Tasks tab and group boards are built on the phone from task and task_update messages, includin
g the user's own Message yourself chat (chat type self).
Never commit secrets; use Firebase config and Secret Manager.
Match the prototype (docs/prototype) for layout, wording and behaviour unless the spec says otherw
ise.
Seed and reset with scripts/seed.ts; demo accounts use Firebase test phone numbers.
```

12. Risks and Open Questions

Risk	Effect	Mitigation
Translation quality in isiZulu or Afrikaans for demo phrases	A weak line in front of investors	Test every demo script phrase early (M2) and adjust the seed messages
Apple developer enrolment takes longer than expected	No iPhone build for demos	Enrol in week one; use Android for early demos
Network at the demo venue	Translation delay or failure	Mobile hotspot; warm-up messages; prototype as fallback
Investors assume the alpha is end-to-end encrypted	Credibility	Say plainly what the alpha does and when end-to-end encryption arrives (section 9)
Cold start delays the first translation after a pause	Translation misses the 2-second demo target	One warm instance on the proxy, matching and onMessageCreate (section 4)
Scope creep beyond the demo scripts	Slower alpha	Build only what the demo scripts and acceptance tests need

Open questions

1. Target date for the first investor demo, to set the milestone pace.
2. Which launch partners besides Uncle Tony’s appear in the strip.
3. Who the 20 alpha users are, and which languages they read, so translation is tested with real pairs.
4. Whether demos need a web version for people without the app, or whether the clickable prototype is enough.
5. How chat history, personal tasks and encryption keys move to a new phone in the pilot; the specification does not cover this yet.

13. Approval and Sign-off

By signing below, the reviewers confirm that this plan is approved as the basis for building the Yttr alpha.

Role	Name	Signature	Date
Product owner	Tony Forbes		
Technical lead	[Name]		